Bayesian neural networks: predictions, uncertainty, and active learning

Teacher's guide and worked solutions

Bayesian machine learning | Skoltech

A neural classifier gives a probability vector at an input. A Bayesian model also represents uncertainty over its parameters and averages predictions over that uncertainty. This seminar uses Monte Carlo dropout as a practical approximation, separates predictive entropy from model disagreement, and uses disagreement to decide which labels to request next.

Tasks 1–3 form a 90-minute core on a one-dimensional example. Task 4 is a separate 45–60-minute GPU practical on MNIST active learning. The image model, training loops, and plotting are supplied. Students implement the calculations that connect model draws, probabilities, uncertainty scores, and acquisition. The companion student and solutions notebooks share their lesson order; this guide provides derivations, worked answers, hints, and interpretation. An additional, fully worked demonstration compares dropout with an explicit Gaussian weight posterior fitted by IVON. It adds no student implementation task and can be shown from its executed figures after the timed material.

The central distinction is between a probability estimated accurately under an approximate model and a probability that is accurate for the real data. More dropout draws reduce Monte Carlo error for the fitted approximation. They do not repair its posterior approximation, training error, or a mismatch between the model and the data-generating process.

1 Teaching plan

1.1 The 90-minute core

The clock includes exercise time. Use the supplied optimizer and figures without explaining every line of their implementation. Keep the connection from posterior averaging to the label-acquisition decision visible throughout.

Clock	Topic	Student activity and intended result
0–10	Posterior and toy data	Identify the two sources of prediction randomness and inspect observed intervals and the gap.
10–20	MC dropout	Distinguish stochastic model draws, deterministic evaluation, and coherent function curves.
20–35	Task 1: predictive probabilities	Transform logits to probabilities before averaging; verify shapes and model-mode restoration.
35–45	Entropy and disagreement	Work through two predictive distributions with the same mean and different disagreement.
45–60	Task 2: uncertainty decomposition	Calculate categorical entropy, expected conditional entropy, and mutual information.
60–70	BALD and the oracle	Derive the information score and identify which labels the acquisition code may access.
70–83	Task 3: active acquisition	Choose unique pool indices; compare label budgets and controlled acquisition runs.
83–90	Interpretation	Explain the finite Monte Carlo limit and distinguish an illustrative result from a general acquisition guarantee.

Before class, run the complete solutions notebook on the actual classroom machine. The core uses PyTorch, NumPy, Matplotlib, and notebook display tools; its data are generated locally and it runs on CPU. The image extension also needs compatible TorchVision, CUDA, and a cached MNIST dataset. Keep the executed solution figures available if classroom training takes longer than expected. Students need Bayes' rule, categorical probabilities, logarithms, expectations, and basic PyTorch tensor operations.

Protect the active-learning endpoint. Start the entropy implementation by minute 45 and discuss the oracle by minute 60. If a function is unfinished, explain its missing mathematical operation and continue with the working solution. Shorten optimizer mechanics and extra architecture experiments before removing the uncertainty decomposition or the budget comparison.

For a 60-minute version, allocate 0–8 minutes to posterior prediction and the toy geometry, 8–20 to Task 1, 20–35 to Task 2, 35–47 to BALD and Task 3, 47–57 to the controlled comparison, and 57–60 to closing questions. Show already executed training results. Keep the nonlinear averaging step, the entropy-versus-disagreement example, and the no-label-peeking acquisition rule.

1.2 The separate GPU practical

Allow 45–60 additional minutes for Task 4. Allocate approximately 10 minutes to the multiclass model and data split, 10–15 to minibatch MC prediction and its checks, 15–20 to the supplied acquisition experiment, and 10–15 to learning curves, acquired images, and paired comparisons. Run the complete experiment before class. This teaching allocation is not a hardware-independent promise about training runtime.

The supplied IVON comparison is a separate demonstration. Allow approximately 5–10 minutes to explain what is sampled, inspect the paired prediction plots, and compare predictive scores. Show the already executed results when the timetable is full. The four implementation tasks and their time allocations do not depend on completing this demonstration.

1.3 Board notation

Object	Meaning
w, $p(w \mid \mathcal{D})$	Neural-network weights and their Bayesian posterior.
$q_\phi(w)$	Approximation used to generate stochastic model draws.
$p_c(x, w)$	Conditional class probability for class c at fixed weights.
$\bar{p}_c(x)$	Probability averaged over the fitted weight approximation.
$\Pi \in \mathbb{R}^{T \times N \times C}$	T model draws, N inputs, and C classes; the class probabilities sum to one along the last axis.
H_{pred}, H_{cond}	Entropy of the mean probability vector and mean conditional entropy across model draws.
$\mathrm{I}_q(Y; W \mid x)$	Model-relative disagreement, $H_{\text{pred}} - H_{\text{cond}}$.
$\mathcal{L}$, $\mathcal{U}$	Labeled training set and remaining unlabeled pool.

All entropies in the notebooks use natural logarithms, so their unit is nats. Do not compare their numerical values with bit entropies without conversion.

2 Purpose and practical limitations: teacher background

Use one short motivation in the timed introduction. The full discussion is preparation material and does not add another block to the 90-minute core. When labels are costly and unlabeled inputs are plentiful, active learning turns uncertainty into a proposed annotation decision. A useful experiment asks whether the chosen rule achieves a specified metric with fewer queried labels under an explicit budget and comparison protocol.

Gal, Islam, and Ghahramani demonstrate a dropout-based image active-learning framework on MNIST and an ISIC2016 lesion-image dataset [4]. These published examples motivate uncertainty-aware label selection. The seminar's compact MNIST experiment is a controlled teaching study, not a reproduction of their architecture, optimization, or results.

High predictive entropy can reflect disagreement among models or ambiguity within each model. BALD targets the former under its fitted distribution. Neither score guarantees detection of out-of-distribution inputs, calibration, or improved accuracy after a particular query. A model can confidently agree on a wrong extrapolation. An informative label can also be atypical of the future test population. Check the evaluation distribution, label quality, class composition, and acquisition cost before interpreting a score as value.

Increasing MC samples, acquiring new labels, and improving optimization act on different uncertainties. The first improves numerical integration under one fitted approximation. The second changes the available training information. The third changes how the approximation is fitted. Keep these operations separate when explaining the observed curves.

3 Bayesian neural-network prediction

3.1 A discriminative model conditions on its inputs

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$. For a neural network with logits $a_w(x) \in \mathbb{R}^C$, define

$$p(y = c \mid x, w) = \mathrm{softmax}(a_w(x))_c.$$

For two classes, a single logit $a_w(x)$ equivalently gives $p(y = 1 \mid x, w) = \sigma(a_w(x))$, with $\sigma(a) = 1/(1 + e^{-a})$. Conditional independence of labels gives

$$p(\mathcal{D} \mid w) = \prod_{i=1}^{n} p(y_i \mid x_i, w).$$

We condition on the inputs. Neither maximum likelihood nor this posterior calculation requires assuming that inputs are uniformly distributed. A prior over weights yields

$$p(w \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid w)p(w)}{p(\mathcal{D})}, \qquad p(\mathcal{D}) = \int p(\mathcal{D} \mid w)p(w)\, dw.$$

The posterior can have many coupled, non-Gaussian directions. Its integrals are generally unavailable in closed form even when the network evaluates quickly at a specified weight vector.

3.2 Average probabilities after the nonlinear network

The posterior predictive distribution is

$$p(y = c \mid x, \mathcal{D}) = \int p(y = c \mid x, w)p(w \mid \mathcal{D})\, dw.$$

Approximate the posterior by q_ϕ and draw $w_t \sim q_\phi$ independently:

$$\bar{p}_c(x) = \mathbb{E}_{q_\phi}[p(y = c \mid x, W)], \qquad \widehat{p}_c(x) = \frac{1}{T}\sum_{t=1}^{T} p(y = c \mid x, w_t).$$

For a fixed fitted approximation, this sample mean is unbiased for $\bar{p}_c$ and consistent as T grows. The Bayesian posterior predictive may still differ from $\bar{p}_c$ because q_ϕ is approximate.

The sigmoid, softmax, and neural network are nonlinear. In general

$$\mathbb{E}_q[\sigma(a_W(x))] \neq \sigma(\mathbb{E}_q[a_W(x)]).$$

For logits -4 and 0 with equal weights, the mean class-1 probability is approximately 0.2590; applying the sigmoid to their mean logit gives $\sigma(-2) \simeq 0.1192$. Averaging hard class decisions discards conditional probability information as well. A deterministic dropout-off evaluation is another prediction rule, not generally the MC predictive mean.

There are two levels of randomness: first draw a model W, then draw the label Y from that model's categorical probabilities. Conditional model probabilities can vary across weight draws even when no label is sampled. For a fixed input, the standard error of their numerical mean is approximately $s_p/\sqrt{T}$. Their spread s_p and this standard error answer different questions. Increasing T shrinks integration error; it does not concentrate the fitted q_ϕ around one network.

4 MC dropout and its approximation assumptions

4.1 What changes between training and prediction

Dropout multiplies selected activations by a random mask. With drop probability r, inverted dropout uses mask entries $M_j = B_j/(1-r)$ with $B_j \sim \text{Bernoulli}(1-r)$. Thus $\mathbb{E}M_j = 1$, while each mask defines a different subnetwork. The trained numerical weights remain fixed during MC prediction; the masks supply the model draws.

Gal and Ghahramani connect dropout training with an approximate variational Bayesian construction [2]. This motivates stochastic prediction with dropout active. It does not make arbitrary dropout settings, priors, and regularizers an exact posterior inference procedure. A Bayesian interpretation requires a specified likelihood and prior, a matching approximating family, and attention to regularization and loss scaling.

In a generic variational formulation,

$$\log p(\mathcal{D}) = \text{ELBO}(q) + \text{KL}(q(w) \| p(w \mid \mathcal{D})),$$

where

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(\mathcal{D} \mid W)] - \text{KL}(q(w) \| p(w)).$$

A dropout training objective uses stochastic likelihood evaluations and a regularization surrogate related to the prior term under the approximation's assumptions. The seminar uses regularized dropout as a practical posterior approximation. It does not claim an exact KL computation for a Bernoulli point-mass weight distribution against a continuous Gaussian prior.

Changing dropout probability, architecture, or the regularization strength changes the fitted approximation. More dropout does not guarantee better uncertainty, and a bigger predictive spread need not imply better calibration. Retrain after changing such settings; repeated prediction alone cannot fit new parameters.

With a mean-loss objective, holding the L2 coefficient fixed while the labeled set grows changes its equivalent strength in the summed objective. A fixed Bayesian prior interpretation would need consistent likelihood and regularizer scaling. The controlled acquisition experiments hold the announced training policy equal across strategies; they are not an exact sequential posterior update under a calibrated prior.

4.2 A model draw must be consistent for a function plot

To plot $x \mapsto p(y = 1 \mid x, w_t)$ as one random function, use the same model realization at every plotted input. Sample the masks once for that draw and reuse them across the grid. If the grid is split into chunks, keep the masks fixed across chunks too. Then resample masks for the next function.

Ordinary dropout often draws separate masks for different rows of one batch. This gives appropriate marginal stochastic predictions at each input, but joining those rows does not display one sampled function. Independently resampled masks per input can turn an otherwise smooth curve into artificial point-to-point noise. The core uses shared realizations for its function plots.

Pointwise entropy and MI require the marginal model distribution at each input; they do not require coupling different inputs to the same model draw. The image practical uses independent per-example

masks for these marginal scores. It consequently does not estimate joint information for a batch of images.

4.3 Prediction modes and state restoration

Use gradient-free inference and restore the model's original mode after sampling. Merely calling `model.train()` can activate unrelated training behavior. In particular, batch-normalization layers can update their running statistics. The supplied helpers separate dropout activation from other training state and restore all relevant flags afterward. The core's shared mask mechanism and the CNN's ordinary dropout mechanism have different purposes; apply the appropriate helper to each.

5 The requested one-dimensional example

The geometry comes from the first example of the supplied 2025 uncertainty seminar. Preserve its four intervals and 30 points per interval:

Class	Intervals	Observations
0	$[-5, -3]$ and $[1, 2.5]$	$30 + 30$
1	$[-3, -2]$ and $[2.5, 5]$	$30 + 30$

There are 120 rows. Inclusive interval endpoints put two observations with opposite labels at $x = -3$ and at $x = 2.5$. These are deliberate consequences of retaining the original construction, not a reason to silently drop rows. The interval $(-2, 1)$ contains no observations.

For evaluation, the notebook separately specifies a population: choose one of the intervals uniformly, sample x uniformly inside it, and use that interval's class. The deterministic 120-point training grid is not itself a random sample from that population. Shared endpoints have population probability zero. The population has no mass in the gap, so a ground-truth conditional label probability there is not identified by this definition. The network still produces a conditional prediction at such an input.

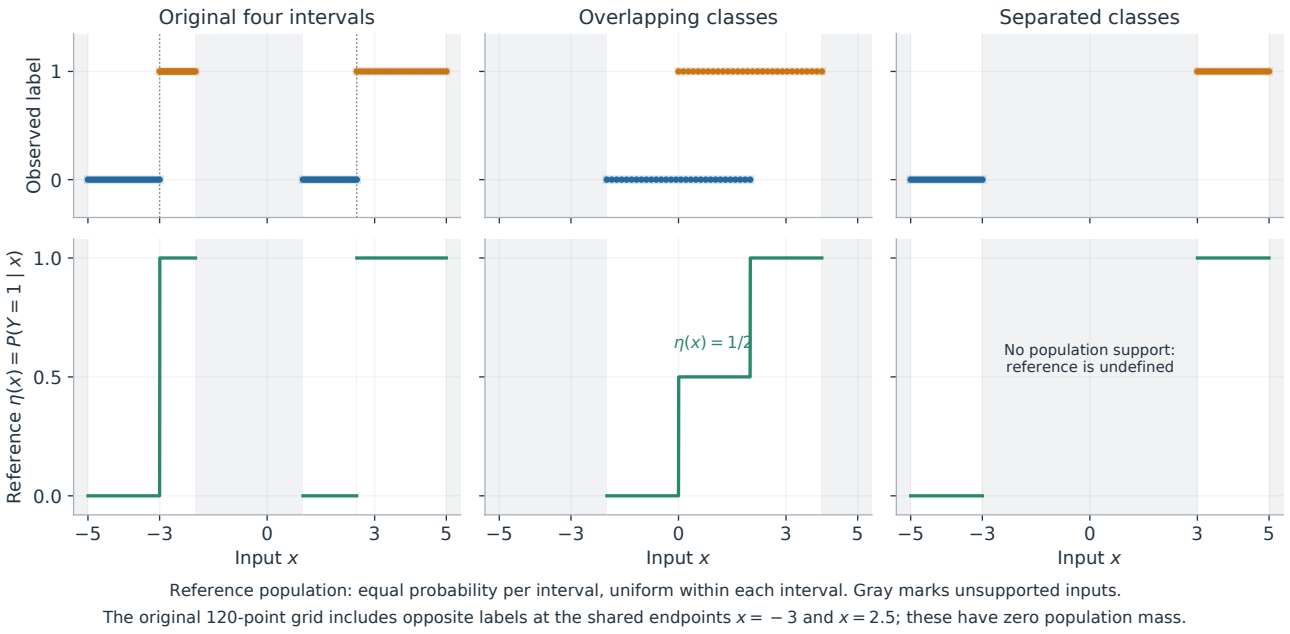


Figure 1: The retained four-interval design and two optional variants. Reference probabilities belong to an explicitly specified mixture-of-intervals population. Gray indicates inputs outside that population's support; the reference conditional probability is left undefined there.

Inspect predictions near observed interiors, the shared boundaries, the gap, and beyond the observed range. Observed conflicting labels at the shared endpoints and missing observations in the gap are different properties of the training design. The fitted uncertainty curves are evidence about this network approximation; they need not respond perfectly to either property. In particular, an empty gap does not mathematically force MC-dropout disagreement to be high.

6 Task 1: posterior predictive probabilities

6.1 What students should do

Complete the function

```
mc_probabilities(model, inputs, n_draws=200)
```

Return a tensor with shape (T, N, C): draw, input, class. Use the supplied `sampled_model` context separately for every draw. It enables the appropriate stochastic modules, holds each shared feature mask fixed across the entire input batch, disables gradients, and restores all previous module modes and MC state in a finally block. Apply softmax along the class axis before stacking draws. Do not average inside this function; later cells need the individual probability vectors.

6.2 Worked implementation

```
def mc_probabilities(model, inputs, n_draws=200):
    if n_draws < 1 or len(inputs) == 0:
        raise ValueError("Need at least one draw and one input")
    draws = []
    for _ in range(n_draws):
        with sampled_model(model):
            draws.append(model(inputs).softmax(dim=-1))
    return torch.stack(draws, dim=0)
```

For a class-1 mean, use `draws.mean(dim=0)[:, 1]`. Calling softmax after averaging logits evaluates a different distribution. Returning the mean alone would also discard the information needed for the MI calculation.

6.3 The supplied original-architecture experiment

The retained MLP has widths $1 \to 10 \to 50 \to 20 \to 1$, sigmoid hidden activations, and dropout probability 0.2 before the sigmoid after the 50-unit layer. For a scalar final logit a, the wrapper returns two logits $(-a/2, a/2)$. This preserves the binary model because

$$\operatorname{softmax}(-a/2, a/2)_1 = \frac{e^{a/2}}{e^{-a/2} + e^{a/2}} = \sigma(a).$$

Cross entropy on these centered logits is the corresponding stable binary logit loss. Preserve the fixed transformation from the original input range $[-5, 5]$ to $[0, 1]$; do not refit normalization from each acquired subset. This uses a known input range, not hidden candidate labels.

The supplied reference fit uses 4,000 full-batch Adam steps, learning rate 0.01, weight decay 10^{-4}, and a OneCycle schedule with maximum learning rate 0.1. Training is supplied so that the exercise focuses on prediction. The main prediction experiment uses 200 model draws. It shows coherent sampled probability functions, their average, and deterministic dropout-off evaluation. These are different objects even when their curves happen to lie close together.

The pointwise 5–95% range is an empirical range of model probabilities across dropout draws. It is not a label interval, an interval for the Monte Carlo mean, or a simultaneous coverage band for the

whole function. The probability mean's MC standard error is calculated separately. Neither range is a guarantee of calibrated Bayesian posterior coverage.

6.4 Checks, hints, and interpretation

Check the full (T, N, C) shape, normalization along classes, finite values, absence of gradient history, and restoration of model state. A zero-dropout network gives the same probabilities in every draw. A stochastic network should vary across independent draws. Repeated inputs within one shared-mask draw must agree; this distinguishes a coherent model function from independent per-row perturbations.

If a student averages logits, use the two-logit numerical example from the board. If all stochastic curves coincide, inspect whether masks were refreshed between draws. If curves are noisy from one input to the next, inspect whether one mask was reused within the draw. A graph of attractive curves alone does not verify the posterior approximation.

A suitable interpretation states that probabilities are averaged after the nonlinear model, the dropout-off curve need not equal that mean, and more MC draws improve numerical precision rather than refitting the network.

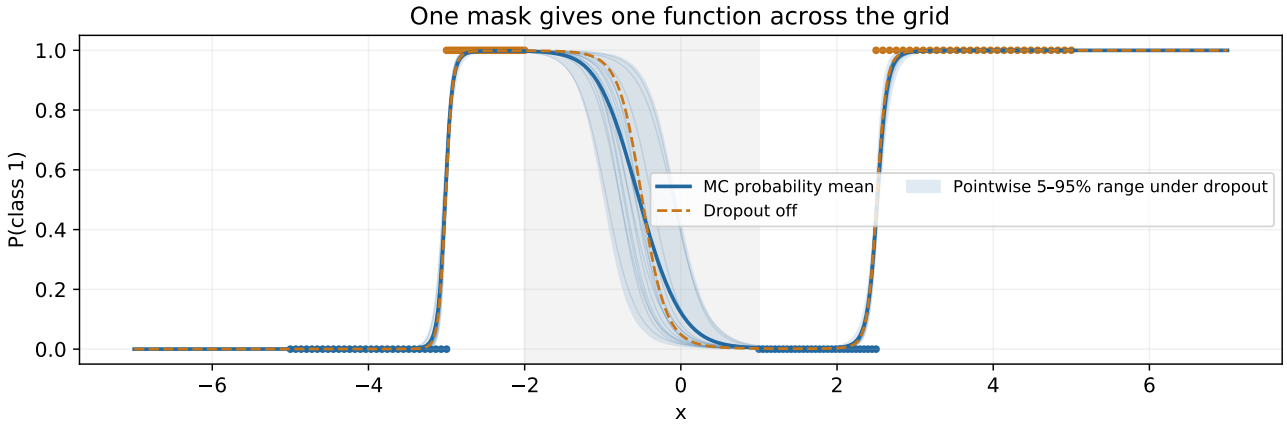


Figure 2: The reused interval example, with coherent dropout functions, their mean and their pointwise probability range. The dashed evaluation curve is a different calculation from averaging stochastic probabilities.

In the executed reference fit, the grid point $x \simeq -1.008$ has class-1 mean probability 0.8684, component-probability standard deviation 0.1516, and MC standard error 0.01072 from 200 draws. The last is approximately the second divided by $\sqrt{200}$. At $x \simeq 5.992$, outside the training range, the model is very confident: mean probability 0.99975. This is a useful counterexample to a claim that missing observations automatically produce uncertain extrapolation.

7 Categorical uncertainty and mutual information

7.1 Entropy has a specified random variable

For a categorical probability vector $p \in [0, 1]^C$,

$$H(p) = -\sum_{c=1}^{C} p_c \log p_c, \qquad 0 \log 0 := 0.$$

It lies between zero and $\log C$. A point-mass label distribution has entropy zero, while the uniform distribution maximizes it. A probability is a number; entropy is a property of an entire categorical distribution.

For the model mixture at input x, define

$$H_{\text{pred}}(x) = H(\mathbb{E}_q[p(\cdot \mid x, W)]),$$
$$H_{\text{cond}}(x) = \mathbb{E}_q[H(p(\cdot \mid x, W))],$$
$$\mathrm{I}_q(Y; W \mid x) = H_{\text{pred}}(x) - H_{\text{cond}}(x).$$

The first is total label uncertainty under the fitted mixture. The second is uncertainty remaining when a model draw is known. The difference measures how much learning a label can distinguish current model draws.

These are often called total, aleatoric, and epistemic uncertainty. In the seminar they are model-relative quantities. The conditional term need not identify the true irreducible noise of nature: misspecification, underfitting, and posterior approximation affect it. Agreement among confidently wrong models gives small MI even when their predictions fail on new inputs.

7.2 Same predictive mean, different uncertainty

Consider two equally weighted model draws for binary classification:

Draw probabilities	H_{pred}	H_{cond}	MI
Both $(0.5, 0.5)$	$\log 2$	$\log 2$	0
$(1, 0)$ and $(0, 1)$	$\log 2$	0	$\log 2$
Both $(0.99, 0.01)$	0.0560	0.0560	0

The first two cases have the same predictive probability $(0.5, 0.5)$ and entropy $\log 2 \simeq 0.6931$. In the first, both models predict an ambiguous label. In the second, each predicts confidently, but they disagree about the label. Predictive entropy alone cannot distinguish these explanations.

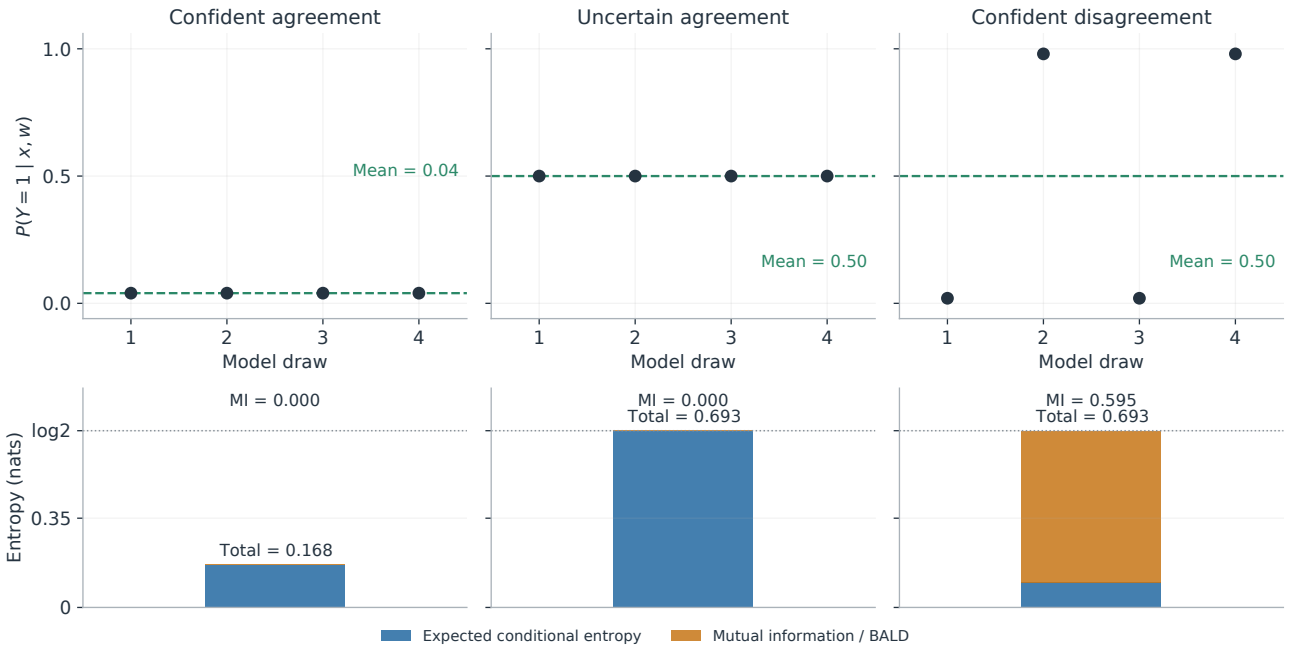


Figure 3: Agreement and disagreement can have different uncertainty interpretations. A mixture of confident, conflicting predictions can have the same mean as predictions that are individually ambiguous. The displayed decomposition is under the chosen model distribution.

7.3 Finite Monte Carlo calculations and bias

Given $\Pi_{tic} = p(y = c \mid x_i, w_t)$, calculate

$$\begin{aligned}
\widehat{p}_{ic} &= \frac{1}{T}\sum_t \Pi_{tic}, \\
\widehat{H}_{\text{pred},i} &= H(\widehat{p}_i), \\
\widehat{H}_{\text{cond},i} &= \frac{1}{T}\sum_t H(\Pi_{ti}), \\
\widehat{I}_i &= \widehat{H}_{\text{pred},i} - \widehat{H}_{\text{cond},i}.
\end{aligned}$$

Sum over classes inside entropy; average model draws along the first axis. Retain one score per input. Entropy concavity implies $\widehat{I}_i \geq 0$ in exact arithmetic. A substantially negative value indicates a shape, normalization, or formula error. Tiny negative roundoff can be handled at the reporting boundary; clipping must not hide an incorrect implementation.

For iid draws, $\widehat{p}$ is unbiased and the mean conditional entropy is unbiased for its q expectation. Entropy of the mean is nonlinear. Jensen's inequality gives

$$\mathbb{E}[H(\widehat{p}_i)] \leq H(\mathbb{E}[\widehat{p}_i]) = H(\bar{p}_i),$$

so the predictive-entropy and MI estimates are generally downward biased at finite T. With $T = 1$, the two entropy terms are identical and estimated MI is zero regardless of the fitted model's true disagreement. More draws help both the estimate and acquisition rankings, but do not change q_ϕ.

For the empirical mixture of T model draws, $\widehat{I}_i \leq \min(\log C, \log T)$. Increasing the draw count does not imply that every observed score increases monotonically in one random run.

8 Task 2: entropy, mutual information, and Monte Carlo error

8.1 What students should do

Complete the two functions

```
categorical_entropy(probabilities)
uncertainty_terms(draws)
```

The entropy function sums only over the last axis and supports arbitrary leading dimensions. Handle $0 \log 0 = 0$ without producing NaNs. The summary function accepts valid normalized draws with shape (T, N, C) and requires $T \geq 2$, because it also returns a sample-standard-deviation estimate of the mean's MC error.

Return a dictionary with these exact keys and shapes:

Key	Shape and meaning
`mean_probability`	(N, C), mean class probabilities
`predictive_entropy`	$(N,)$, entropy of those means
`expected_entropy`	$(N,)$, mean entropy of each conditional prediction
`mutual_information`	$(N,)$, difference of the two entropy terms
`probability_mc_se`	(N, C), sample standard deviation divided by $\sqrt{T}$

8.2 Worked implementation

```
def categorical_entropy(probabilities):
    validate_probabilities(probabilities)
```

```
    tiny = torch.finfo(probabilities.dtype).tiny
    return -(probabilities * probabilities.clamp_min(tiny).log()).sum(-1)

def uncertainty_terms(draws):
    if draws.ndim != 3 or draws.shape[0] < 2:
        raise ValueError("Expected (T,N,C) with at least two draws")
    validate_probabilities(draws)
    mean = draws.mean(dim=0)
    predictive = categorical_entropy(mean)
    expected = categorical_entropy(draws).mean(dim=0)
    return {
        "mean_probability": mean,
        "predictive_entropy": predictive,
        "expected_entropy": expected,
        "mutual_information": (predictive - expected).clamp_min(0),
        "probability_mc_se": draws.std(dim=0, unbiased=True)
                             / math.sqrt(draws.shape[0]),
    }
```

The supplied probability validator checks floating dtype, finite entries in $[0, 1]$, at least two classes, and class sums near one. It does not silently renormalize an invalid tensor. The logarithm is evaluated at a positive floor, but the multiplying probability remains the original value, so a zero entry contributes zero. Clipping negative MI here addresses numerical roundoff after correct formulas; the deterministic checks must still expose a wrong axis or sign.

8.3 Worked checks and useful hints

For identical $(0.5, 0.5)$ draws, both entropy terms equal $\log 2$, MI is zero, and the probability MC standard error is zero. For two opposite deterministic draws, the predictive entropy is $\log 2$, the expected entropy is zero, and MI is $\log 2$. The unbiased standard deviation of either class probability is $1/\sqrt{2}$, so its MC standard error at $T = 2$ is $1/2$. The notebook's four-draw counterpart has each deterministic vector twice; its MC standard error is $1/\sqrt{12} \simeq 0.2887$. A high disagreement score can coexist with a noisy numerical mean when the draw count is tiny.

If the wrong axis is averaged, first label the axes as draw, input, and class. If zeros create NaNs, isolate one deterministic probability vector and inspect its entropy. A uniformly small MI can reflect posterior approximation or a small MC budget; it is not automatically a coding error. The helper rejects $T = 1$ for its MC-error contract, although the separate algebraic statement that a one-draw MI estimate is zero remains true.

For interpretation, ask students to compare predictive entropy and MI at an observed interval interior, a shared training endpoint, and the unsupported gap. Require a description of the observed fitted curves. Do not require the approximation to express a predetermined uncertainty shape.

At $x \simeq -1.008$, the reference values are predictive entropy 0.38944, expected conditional entropy 0.30524, and MI 0.08420 nats. At the extrapolated point $x \simeq 5.992$, MI is only 3.15×10^{-6} nats. The same network can therefore express disagreement in part of a gap and almost none in another unsupported region. This is evidence about its approximation and inductive behavior, rather than a general rule for detecting unfamiliar inputs.

9 BALD: labels that distinguish current hypotheses

9.1 A posterior information objective

BALD is Bayesian Active Learning by Disagreement. Houlsby et al. express a parameter-information objective using predictive label entropies, avoiding explicit high-dimensional posterior-entropy calcula-

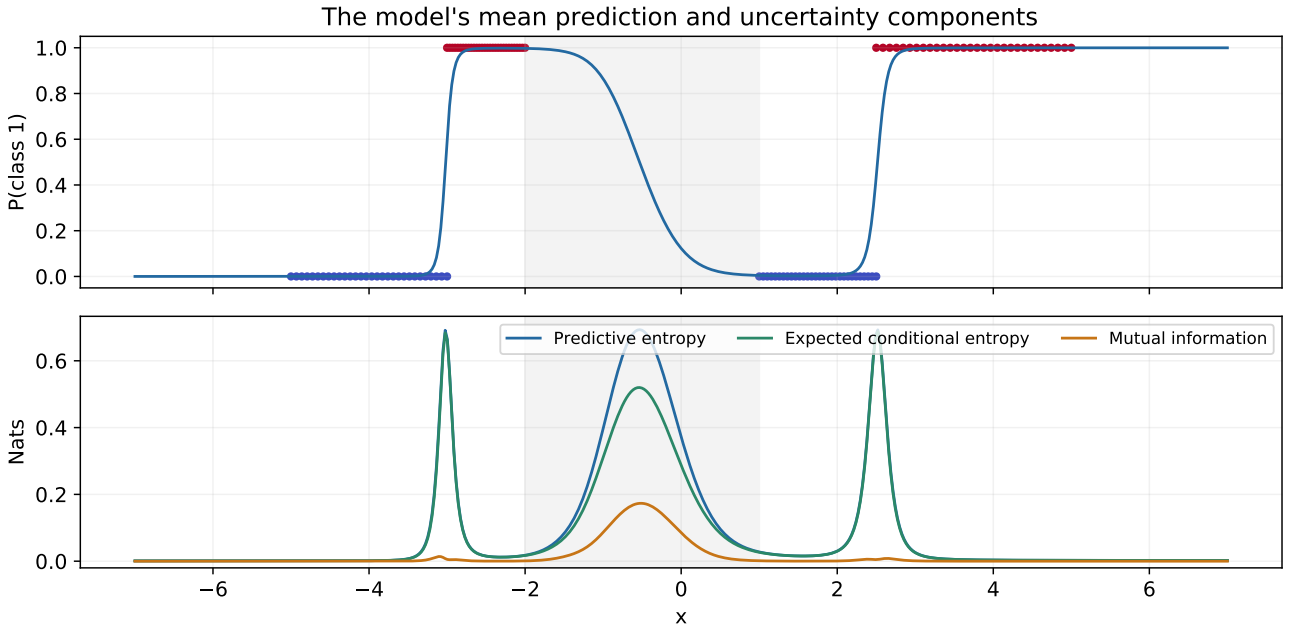


Figure 4: Predictive entropy, expected conditional entropy and disagreement for the original fitted network. The gray region has no observations; the decomposition describes the fitted approximation, not a known label distribution inside that region.

tions [3]. For an exact Bayesian model, the desired score is

$$a(x) = \mathrm{I}(Y; W \mid x, \mathcal{D}).$$

One form is the expected information gained by updating the weight posterior:

$$a(x) = \mathbb{E}_{Y|x,\mathcal{D}}[\mathrm{KL}(p(w \mid \mathcal{D}, x, Y) \| p(w \mid \mathcal{D}))].$$

When the differential entropies exist and are finite, it also equals

$$H(W \mid \mathcal{D}) - \mathbb{E}_{Y|x,\mathcal{D}}[H(W \mid \mathcal{D}, x, Y)].$$

The expectation is over an as-yet unobserved label. The acquisition procedure can evaluate the score before requesting that label.

9.2 Derivation in the small label space

For fixed x and a weight distribution q, define $\bar{p}_c = \int q(w)p(c \mid x, w)\, dw$. The joint distribution is $q(w)p(c \mid x, w)$. Expanding mutual information gives

$$\begin{aligned}
\mathrm{I}_q(Y; W \mid x) &= \sum_c \int q(w)p(c \mid x, w) \log \frac{p(c \mid x, w)}{\bar{p}_c}\, dw \\
&= \mathbb{E}_q \left[\sum_c p(c \mid x, W) \log p(c \mid x, W) \right] - \sum_c \bar{p}_c \log \bar{p}_c \\
&= H(\bar{p}) - \mathbb{E}_q[H(p(\cdot \mid x, W))].
\end{aligned}$$

Only entropies of C-class label vectors are required. No entropy of a millions-dimensional weight vector is evaluated by the code.

For the approximate distribution q, define its hypothetical one-label update by

$$q_c(w \mid x) = \frac{q(w)p(c \mid x, w)}{\bar{p}_c}, \qquad \bar{p}_c > 0.$$

The same score equals $\sum_c \bar{p}_c \, \mathrm{KL}(q_c \| q)$. This expression remains meaningful for discrete dropout model draws, whose differential weight entropy would not be an appropriate quantity. Retraining a restricted network approximation after acquisition need not produce exactly this hypothetical update. The score is a model-based decision rule, not a guarantee that the measured uncertainty or test error will decrease after every query.

9.3 Selecting several points is an additional approximation

Ranking marginal BALD scores and taking the largest k maximizes their sum under a fixed-cardinality constraint. It does not generally maximize joint information $\mathrm{I}(Y_{x_1}, \ldots, Y_{x_k}; W \mid \mathcal{D})$. Correlated or nearly duplicate candidates may be redundant. This seminar uses the transparent marginal-score rule and reports the acquisition batch size; it does not claim to implement joint BatchBALD.

10 The active-learning protocol

10.1 The oracle boundary and label ledger

The acquisition code receives candidate inputs, model predictions or scores, and a requested count. It must not inspect candidate labels. After indices are chosen, the oracle reveals only those labels. Add the queried rows to the labeled set and remove their indices from the pool.

Track unique indices and use random acquisition without replacement. An unlabeled observation must not be requested twice. With n_0 initial labels and acquisitions $k_1, \ldots, k_r$, the training label budget is

$$B_r = n_0 + \sum_{j=1}^{r} k_j.$$

Count the initial labels too. Limit each request by both the remaining label budget and the pool size. Stop before requesting labels when the budget is exhausted. Separately report any already available validation labels used for evaluation; they are a real labeled resource even if excluded from the training acquisition budget.

An offline dataset contains hidden labels somewhere in memory. This alone does not invalidate an active-learning simulation. The enforceable boundary is which functions receive or read them. Test a selector with the same inputs and different hidden labels: acquisition indices must remain unchanged. Labels used to audit selected class composition are inspected only after queries, not used to choose them.

10.2 Keep the comparison controlled

Within a paired replicate, use the same initial labeled indices, candidate pool, model initialization policy, architecture, dropout probability, optimizer schedule, training budget, and evaluation draw count for all acquisition methods. The acquired inputs then differ by design. Use independent random seeds across paired replicates, and preserve a record of the queried indices and actual label counts.

State whether each round starts from the same initialization or continues from the previous model. Either can define a useful experiment; applying different policies to acquisition methods confounds their comparison. A fixed number of epochs and a fixed number of optimizer updates are distinct computational budgets when the labeled set grows. The notebook specifies one policy and applies it equally to the methods.

Use validation data only for the announced monitoring or model-selection purpose. Keep test data outside the acquisition and tuning loop. Record validation curves against the number of acquired labels, not only the round number, and evaluate the held-out test set at the planned final point.

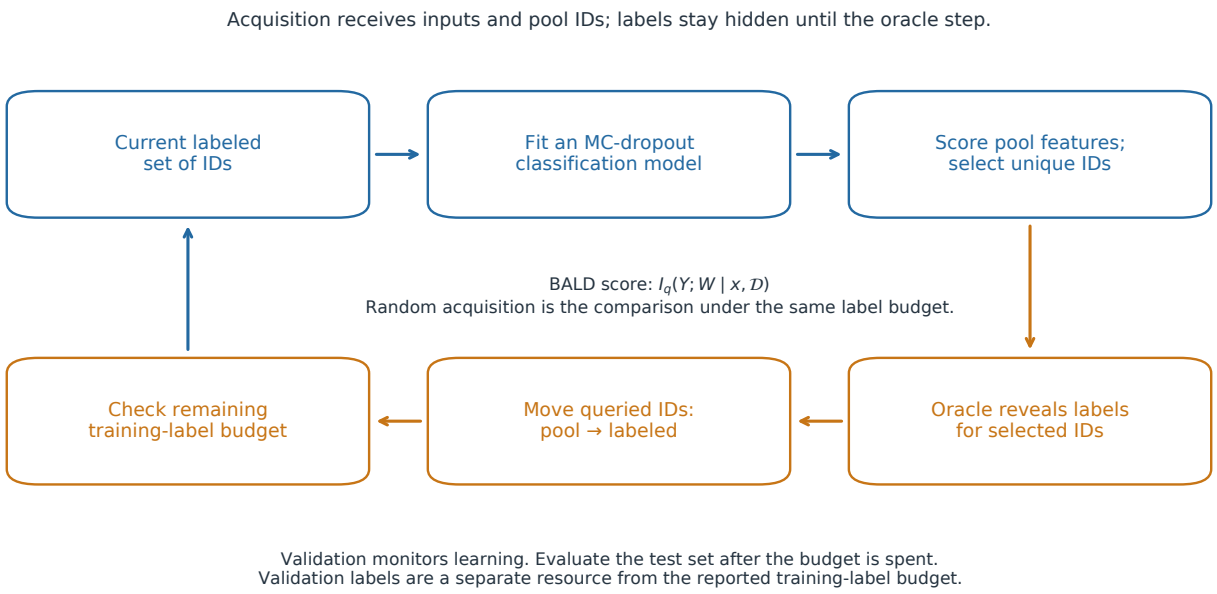


Figure 5: Pool-based active learning separates input-only scoring from label revelation. Query only after acquisition indices have been selected; move queried points out of the pool and include their labels in the ledger.

10.3 Read repeated results as a small controlled experiment

Report random acquisition alongside predictive entropy and BALD when available. Compare their accuracy and predictive negative log likelihood at equal label budgets. A larger accuracy and a smaller NLL are preferable under those metrics. They need not rank models identically: NLL also evaluates confidence assigned to the observed labels.

For replicate r, form a paired endpoint difference, such as $\Delta_r = \mathrm{Acc}_{\mathrm{BALD,r}} - \mathrm{Acc}_{\mathrm{random,r}}$. Average the paired differences and report their variation. Several repeated seeds are more informative than one winning curve, but three small trials still do not establish a universal ranking. Variation across test images with models fixed is different from variation across training/acquisition runs. Avoid treating every image as an independent acquisition experiment.

11 Task 3: acquire pool indices without seeing labels

11.1 What students should do

Complete the selector

```
acquisition_indices(pool_ids, draws, n_query, strategy="bald", generator=None)
```

Return unique global pool IDs, not local positions in the supplied candidate array. Strategies are random, predictive entropy, and BALD. The function has no label argument. For score ties, prefer the smaller global ID, independent of the order of the candidate array. A random request samples without replacement using the supplied generator.

The contract assumes unique input IDs and valid probability draws in the same candidate order. Reject negative requests and requests larger than the number of candidates. A zero request returns an empty tensor of IDs. Validate these conditions before performing any oracle update.

11.2 Worked selection logic

For random selection, choose local positions from a random permutation and map them back to global IDs. For entropy or BALD, compute the corresponding pointwise score. Sort global IDs first and then perform a stable descending score sort. Stability preserves increasing global-ID order within equal scores.

```
if strategy == "random":
    positions = torch.randperm(len(pool_ids), generator=generator)[:n_query]
else:
    terms = uncertainty_terms(draws)
    key = "predictive_entropy" if strategy == "entropy" else "mutual_information"
    scores = terms[key]
    by_id = torch.argsort(pool_ids, stable=True)
    ranking = torch.argsort(scores[by_id], descending=True, stable=True)
    positions = by_id[ranking[:n_query]]
selected_ids = pool_ids[positions]
```

This is the core selection step; the notebook's full implementation also checks allowed strategy names, shapes, input uniqueness, and requested count. The supplied `LabelOracle.query` rejects duplicate IDs, previously queried IDs, and requests over its remaining budget atomically. A rejected request must leave both its query history and label count unchanged. It also rejects floating-point and Boolean IDs before conversion, avoiding an accidental request for ID 0 when a caller passes 0.9 or False.

11.3 The controlled one-dimensional acquisition experiment

The 120 retained design rows become a finite pool. For each of three paired replicates, query 12 random initial IDs without inspecting their labels. Then make four requests of six labels. The actual budgets are 12, 18, 24, 30, and 36; all initial and new oracle requests count.

Compare random acquisition, predictive entropy, and BALD. Each budget starts a fresh model and trains it for 1,000 full-batch steps.

The supplied small-data network has widths

$$1 \to 32 \to 32 \to 2.$$

It uses two tanh hidden layers, dropout probability 0.2 after the first hidden layer, and input scaling $x/5$. Adam uses weight decay 10^{-4} and a OneCycle schedule with maximum learning rate 0.03, division factor 10 and final division factor 100. This smaller network helps avoid an underfitted constant predictor on the initial 12 labels. The original sigmoid network and its 4,000-step fit remain the motivating example in Tasks 1 and 2.

Within a replicate and round, methods share the initialization and training schedule; the acquired data are allowed to differ. Evaluation uses 512 independent mixture-of-interval samples for the displayed validation curves and 1,024 independent final test samples. Pool scoring and validation prediction use 64 model draws; final test prediction uses 256. Those evaluation labels are a separately available resource, outside the training-label ledger.

Do not compare an acquisition curve against the fully labeled reference fit as though both used the same number of labels. The reference fit establishes the approximation's behavior on the original full toy design. The active experiment studies the cost of learning from a restricted subset.

The result table includes predictive NLL and a class-summed Brier score:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C}(\widehat{p}_{ic} - \mathbf{1}\{y_i = c\})^2.$$

For binary classification this convention equals twice the squared error of the class-1 probability. State that convention when comparing numbers with other implementations. Accuracy evaluates the chosen class, while NLL and Brier also evaluate the full probability vector.

The executed reference comparison gives:

Strategy	Test accuracy	Repeat SD	Test NLL	Test Brier
Random	0.98145	0.00610	0.03840	0.02401
Entropy	0.83301	0.14208	1.2146	0.33598
BALD	0.83236	0.14265	1.1467	0.33570

All methods use 36 acquired labels, including the initial 12. Values average three paired runs; the standard deviation describes repeat variation, not a confidence interval. Last digits can differ across software stacks.

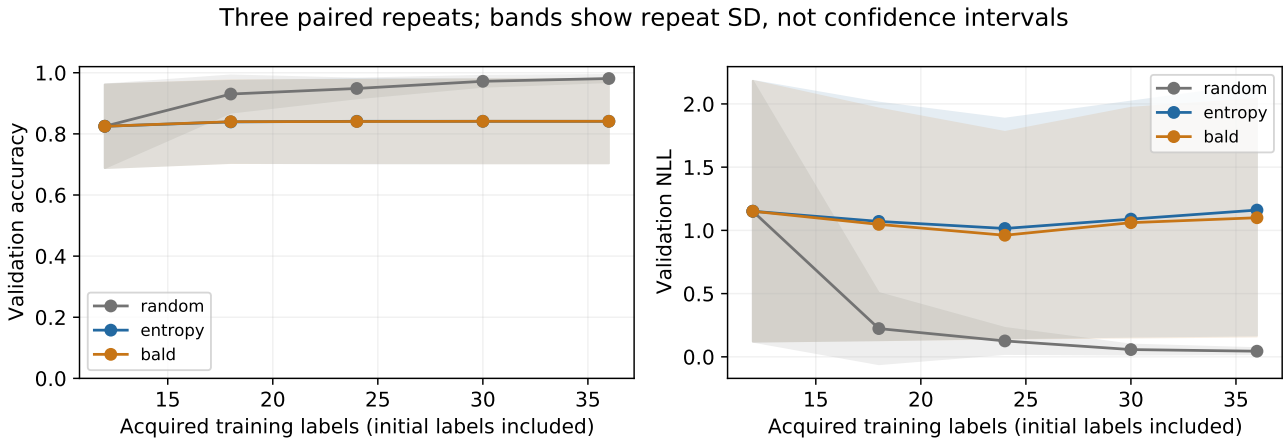


Figure 6: Validation curves at common label budgets. Random acquisition performs better in this particular toy experiment. A correct information calculation does not guarantee an advantageous acquisition sequence under an approximate model and a restricted batch policy.

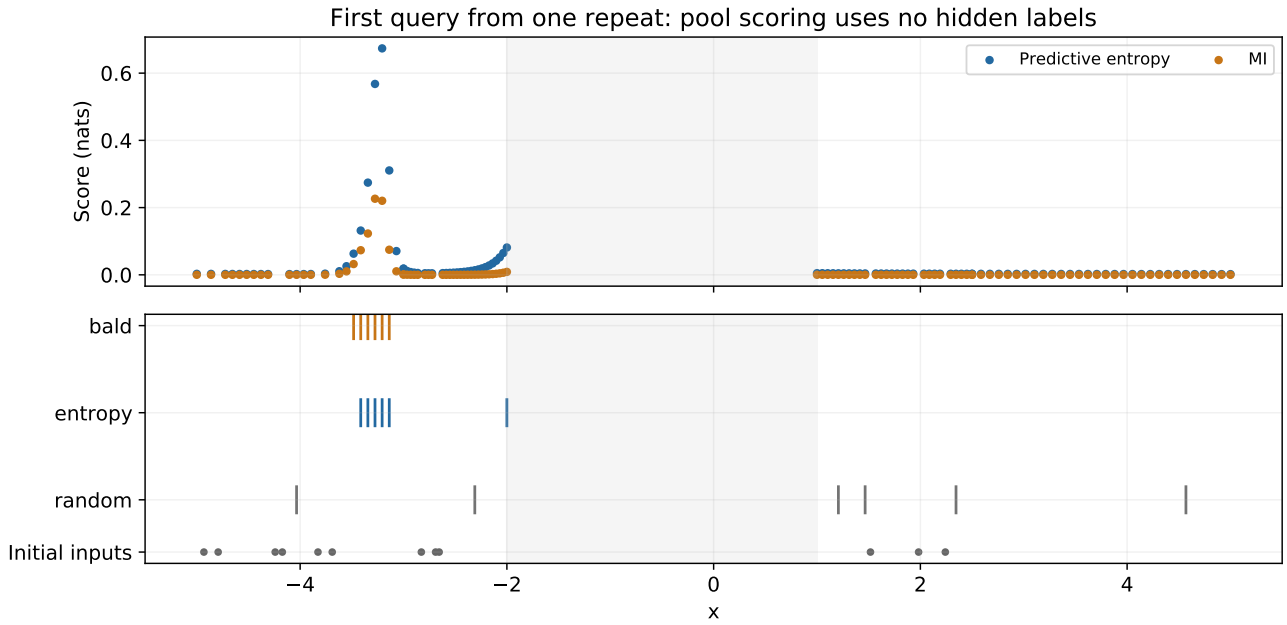


Figure 7: The first query in one repeat. Pointwise entropy and BALD can spend a batch on neighboring inputs near one uncertain transition. Random selection covers a wider part of the pool. Scores are computed before any of the new labels are revealed.

Read this result directly: random acquisition has better mean accuracy, NLL and Brier here. The first-query plot suggests one mechanism for failure: pointwise ranking buys redundant neighbors, while

a confidently wrong model may assign low scores to regions that need exploration. This is not proof that either information criterion is universally inferior. The image experiment below gives a different comparison under a different setting. Before discussing acquisition quality, check that a model can fit its currently labelled observations; optimizer failure and uncertainty quality are separate questions.

11.4 Checks and interpretation answers

Use handcrafted scores to check the ranking, unique IDs, exact requested count, zero-request behavior, and stable tie order. Reordering candidate inputs together with their probabilities must not change a deterministic selection. Two independent random generators with the same seed should reproduce random selection without duplicate IDs. An oracle test must verify that failed requests consume no labels.

A correct implementation can still give a poor acquisition outcome. The score measures model-relative information at the current fit. Approximate training, small samples, batch redundancy, and the future evaluation population all affect whether that information improves accuracy or NLL. A defensible explanation cites the observed repeated-run results and the label budget, rather than declaring BALD a universal winner.

12 Task 4: GPU active learning for MNIST

12.1 Purpose and the fourth implementation

This is a separate 45–60-minute practical. It studies a classifier that predicts one of ten digit labels from an image; it does not fit an image density as in the normalizing-flow practical. The same probability averaging and entropy decomposition now operate on a multiclass CNN. The model, label oracle, training policy, acquisition loop, and plots are supplied.

Complete the function

```
mc_cnn_probabilities(model, images, n_draws=15, batch_size=512)
```

Inputs are uint8 images with shape $(N, 1, 28, 28)$, on CPU or GPU. Return a CPU float32 tensor $(T, N, 10)$ with no gradient history. Preserve image order when joining minibatches and preserve every module's original training mode. The device comes from the model's first parameter. The supplied input helper uses the fixed MNIST normalization $(I/255 - 0.1307)/0.3081$.

12.2 Worked implementation

```
def mc_cnn_probabilities(model, images, n_draws=15, batch_size=512):
    if n_draws < 1 or batch_size < 1 or len(images) == 0:
        raise ValueError("Use positive draws, batch size, and nonempty images.")
    device = next(model.parameters()).device
    draws = []
    with torch.inference_mode(), mnist_dropout_only(model):
        for _ in range(n_draws):
            batches = []
            for start in range(0, len(images), batch_size):
                x = mnist_inputs(images[start:start + batch_size], device)
                batches.append(model(x).softmax(dim=-1).float().cpu())
            draws.append(torch.cat(batches, dim=0))
    return torch.stack(draws, dim=0)
```

The probability vectors are retained, rather than hard labels or averaged logits. Taking their mean over the first axis produces the predictive probabilities used for evaluation. Computing uncertainty requires the draws as well as that mean.

The supplied context evaluates the model, activates only dropout modules, and restores all individual mode flags in a finally block. Standard dropout uses independent masks across images and across minibatches. This is valid for each image's marginal entropy and MI. It does not give a coherent function draw across images and does not estimate joint batch information.

12.3 What the CPU checks establish

A deterministic probe with known logits verifies the output values, not just their shape. An input-sensitive pixel probe independently checks the fixed normalization and image order across minibatches of unequal final size. A stochastic probe verifies variation across draws and that each class vector is finite and normalized. The probe deliberately contains batch normalization and mixed initial module modes. Its running means and variances must not change, and its mode flags must be restored. An interrupted context also checks restoration after an exception.

If stochastic predictions are identical, inspect dropout activation. If predictions differ with minibatch size even when dropout is disabled, inspect batch-normalization mode and input order. A successful probability check does not establish that the trained CNN is calibrated; it verifies the specified prediction calculation.

12.4 The supplied CNN and training policy

The CNN uses two convolutional blocks with 16 and 32 channels, ReLU activations, and two-by-two max pooling. A width-128 hidden layer follows the flattened $32 \times 7 \times 7$ representation. Dropout probability 0.5 precedes the final ten-class linear layer.

At every label budget, construct a fresh model from the prescribed seed and train for 150 Adam updates, batch size 64, learning rate 10^{-3}. The same round-specific seed and policy apply to all methods in a paired replicate. A fixed update count prevents a larger acquired dataset from silently granting more training computation. It is a teaching budget, not a claim of optimizer convergence at every round.

This practical uses cross entropy and stochastic dropout, without an explicitly evaluated weight-posterior KL term. Its uncertainty scores are under the fitted dropout approximation. Do not claim that the optimization computes an exact posterior for a fully specified Gaussian weight prior.

12.5 The label ledger and separate evaluation resource

Select 6,000 candidate images uniformly from the official MNIST training set without consulting their labels. Reserve a disjoint 1,000-image evaluation set from the training source for learning curves. Within each paired repeat, all methods begin with the same 100 uniformly selected initial pool IDs. Each method queries those initial labels from its own oracle.

Four acquisitions of 100 labels produce budgets 100, 200, 300, 400, and 500. The remaining pool shrinks as IDs are purchased, and previously acquired IDs cannot be queried again. Random acquisition, predictive entropy, and BALD all have the same budgets. The candidate pool and initial set are not secretly class-balanced through hidden label inspection.

The 1,000 evaluation labels are a separate available resource and are not charged to the acquisition-training ledger. State that cost explicitly. Their curves do not alter the announced update count, dropout probability, stopping budget, or acquisition method. Only after every acquisition run is finished does the experiment evaluate the official 10,000-image test set.

Three paired replicates share initial labels and model seeds within each replicate and use independent seeds between replicates. Entropy and BALD choose the top 100 marginal scores; no diversity or joint-information correction is applied. Acquisition scoring costs additional computation that random sampling does not need. Equal training updates and label budgets do not imply equal wall-clock cost for the methods.

12.6 GPU setup, caching, and runtime

The CPU checks run without CUDA. Set `RUN_MNIST` to True to enable the practical on a CUDA device. Otherwise the notebook prints an explicit skip message. Install compatible PyTorch and torchvision versions and cache MNIST before class. The default data location is shared with the preceding seminar; results and fitted models use a separate BNN cache.

```
BNN_MNIST_DATA   # default: /tmp/sk_bayes_mnist/data
BNN_MNIST_CACHE  # default: /tmp/sk_bayes_bnn_mnist
MNIST_RETRAIN = True
```

Set the environment variables before running the notebook to use persistent locations. The default performs a fresh experiment; set retraining to False only to replay a matching completed experiment. A cached replay is not a new training measurement. Runtime depends on the GPU and software stack, so keep executed figures available for classroom discussion.

12.7 Accuracy, predictive NLL, and calibration measure different things

For mean probabilities $\widehat{p}_i$, accuracy compares their maximum-probability class with the observed label. Predictive NLL is

$$-\frac{1}{n}\sum_{i=1}^{n}\log \widehat{p}_{i,y_i}.$$

The log is applied after averaging model probabilities. Averaging each subnetwork's cross entropy instead evaluates expected conditional loss, which is generally larger by Jensen's inequality and is not the same predictive metric.

The practical also reports an expected calibration error estimate with 15 equal-width confidence bins:

$$\widehat{\mathrm{ECE}} = \sum_{b}\frac{n_b}{n}|\,\mathrm{accuracy}(b) - \mathrm{meanconfidence}(b)|.$$

Empty bins contribute zero. This is a finite-sample, bin-dependent diagnostic, not a complete calibration theorem or a proper scoring rule. Accuracy can improve while this diagnostic worsens. Compare it with NLL and the number of repetitions instead of treating one scalar as universal model quality.

A finite MC mean introduces numerical variation into these metrics. In particular, $-\log \widehat{p}_{i,y_i}$ is generally upward biased relative to $-\log \bar{p}_{i,y_i}$ at finite draw count. The experiment holds the 15-draw budget fixed across methods. Its three repeats describe variability of the specified training/acquisition experiment, not a broad population of all architectures or acquisition protocols.

12.8 Executed GPU reference

The complete fixed-budget image experiment ran on an NVIDIA L40 in about 16 seconds in the recorded environment. This includes all 45 fits and the final evaluations; it is not a runtime guarantee for another GPU. The full solutions notebook, including CPU experiments and appendices, took about 80 seconds in this run.

Strategy	Test accuracy	Repeat SD	Test NLL	ECE
Random	0.917	0.003	0.303	0.057
Entropy	0.938	0.014	0.277	0.107
BALD	0.946	0.003	0.272	0.120

All methods end with 500 purchased training labels, including the initial 100. Here BALD has better average accuracy and predictive NLL than random, while the binned ECE is worse. This contrast is worth discussing: correct class decisions, good predictive log probabilities, and agreement of confidence with empirical correctness are distinct criteria. These three repeats do not support a universal ordering of acquisition strategies.

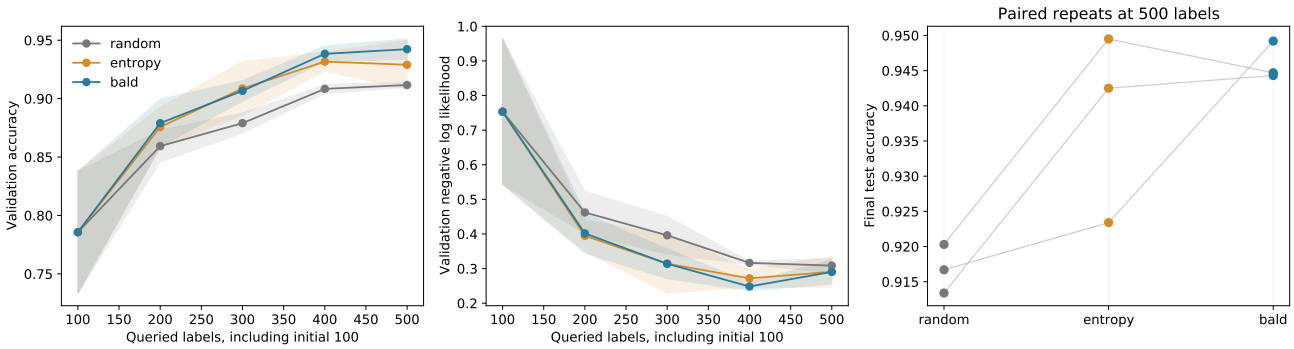


Figure 8: The GPU practical's learning curves and paired final-test outcomes. Common initial labels and model seeds make the endpoint comparisons paired. Bands show repeat standard deviation, rather than a confidence interval.

12.9 Read the figures without using future labels

Learning curves show accuracy and NLL against the actual number of purchased labels. Shaded bands are one standard deviation across the three repeats, not confidence intervals. Connected test endpoints show the pairing between methods that began with the same data and model seed.

At the first acquisition, entropy and BALD start from the same fitted CNN. A scatterplot of predictive entropy against MI, colored by expected conditional entropy, shows how the criteria can differ before any new labels are known. Highlighting the selected BALD batch does not require its labels.

The acquisition image panels display the first eight selected digits for each strategy in the first repeat. Their labels are revealed after selection and then shown as captions. Several similar selected digits can expose the redundancy of marginal batch ranking. A class-count heatmap audits labels already bought; it never examines the remaining hidden pool labels. Class balance is a useful diagnostic, not the objective of either information criterion.

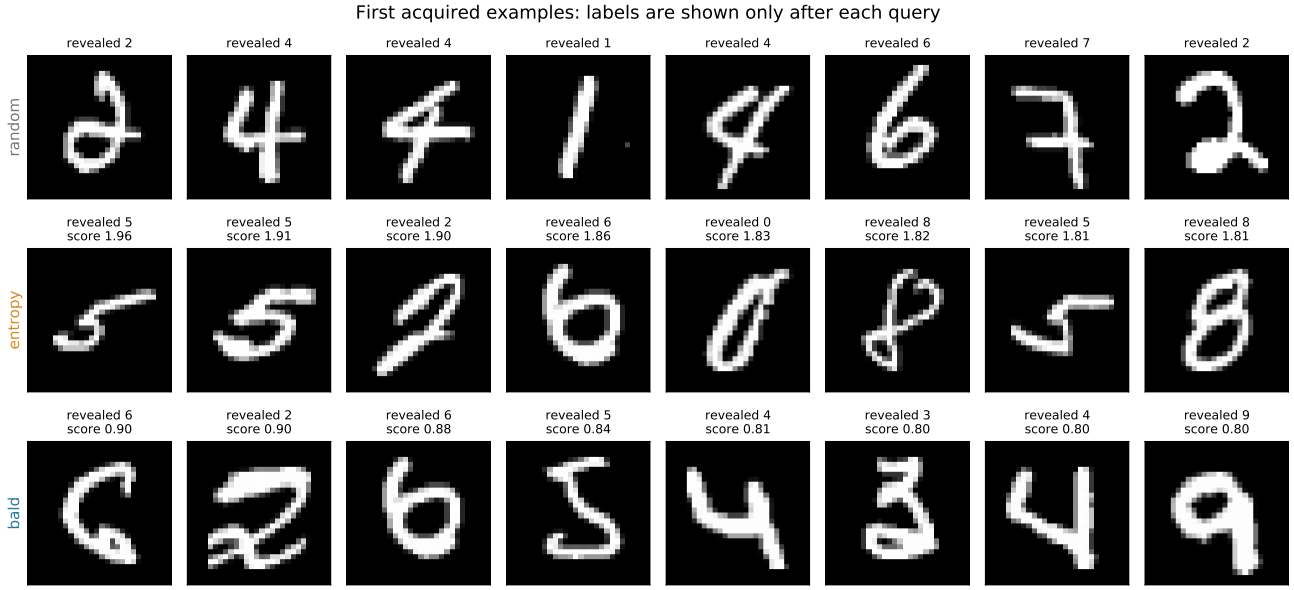


Figure 9: First selected MNIST examples in the first repeat. Labels appear only after they have been purchased. Entropy and BALD may choose different images from the same starting model because their scores measure different properties of its predictive mixture.

12.10 Worked interpretation and questions

A suitable explanation identifies the common budget and model policy before comparing outcomes. It describes whether a ranking persists across paired repeats and whether the observed difference is large relative to their spread. It also separates classification accuracy from confidence quality.

Ask why two criteria select different digits from the same starting CNN, why repeated similar digits can all rank highly, and which resources a label-budget plot omits. Require students to account for evaluation labels and acquisition computation. A planned follow-up can study different batch selection or distribution shift, but the existing test set would then be a development resource if its results influence that choice.

13 Supplied demonstration: explicit variational weights with IVON

13.1 Purpose and the comparison to dropout

This section is a worked example, with all training and prediction code provided in both notebooks. Students do not implement a fifth task. Its purpose is to make the posterior approximation visible: dropout samples subnetworks through activation masks, whereas IVON samples numerical weights from a fitted Gaussian distribution.

Improved Variational Online Newton (IVON) is a variational optimizer introduced in the 2024 ICML paper by Shen and colleagues [1]. It combines updates of the weight means with a diagonal curvature state that determines the posterior variances. The network can retain familiar linear, convolutional, and activation layers. A different optimizer and sampling mechanism make it a Bayesian neural-network approximation; no special "Bayesian activation" is required.

This is a modern optimization method for a comparatively simple posterior family. A diagonal Gaussian cannot represent weight correlations or separate posterior modes. The demonstration compares two fitted approximations and their predictive behavior. It does not establish that either has recovered the exact weight posterior, or that a newer method must give better uncertainty on every dataset.

13.2 The Gaussian approximation and its objective

For a flattened parameter vector with d components, IVON uses

$$q(w) = \prod_{j=1}^{d} \mathcal{N}(w_j; \mu_j, \sigma_j^2), \qquad w_j = \mu_j + \sigma_j \epsilon_j, \quad \epsilon_j \sim \mathcal{N}(0, 1).$$

Means and variances are fitted using training data. Every weight and bias in the demonstration belongs to this approximation; dropout is disabled in the IVON network. At fixed sampled weights, the network is deterministic.

With N observations and a prior $p(w)$, the negative ELBO divided by the dataset size is

$$\mathcal{J}(q) = \frac{1}{N} \mathbb{E}_q \left[\sum_{i=1}^{N} -\log p(y_i \mid x_i, W) \right] + \frac{1}{N} \mathrm{KL}(q(w) \| p(w)).$$

For a zero-mean isotropic Gaussian prior of precision α,

$$\mathrm{KL}(q \| p) = \frac{1}{2} \sum_{j=1}^{d} \left[\alpha(\mu_j^2 + \sigma_j^2) - 1 - \log(\alpha \sigma_j^2) \right].$$

The first term rewards a good fit under sampled weights. The KL term penalizes departures from the prior and prevents variational inference from reducing to an unconstrained search for accurate stochastic

predictions. The displayed KL expression explains the objective; it is not an extra loss term added to the IVON training loop.

The official implementation uses an effective sample size λ, `ess`, and a prior coefficient δ, called `weight_decay`. For its mean-cross-entropy convention,

$$\sigma_j^2 = \frac{1}{\lambda(h_j + \delta)}, \qquad \alpha = \lambda\delta,$$

where h_j is the fitted diagonal curvature state. It is not the exact Hessian of the complete posterior. Setting $\lambda = N$ matches the ordinary dataset likelihood scaling. In the 120-observation example, $\lambda = 120$ and $\delta = 10^{-4}$ imply prior precision 0.012. Changing `ess` changes the effective likelihood count, the posterior spread, and, if δ is held fixed, the implied prior precision as well. It is not the number of Monte Carlo prediction draws. To vary dataset size while holding a chosen prior precision α fixed, set $\delta = \alpha/N$ along with $\lambda = N$.

13.3 Training and coherent prediction draws

The essential official API is short:

```
optimizer = ivon.IVON(
    model.parameters(), lr=chosen_lr,
    ess=len(x_train), weight_decay=1e-4,
)
for step in range(train_steps):
    with optimizer.sampled_params(train=True):
        optimizer.zero_grad(set_to_none=True)
        loss = F.cross_entropy(model(x_train), y_train)
        loss.backward()
    optimizer.step()
```

The context samples weights and collects the stochastic likelihood gradient. Leaving it restores the parameter means; `step()` updates the variational state. Backpropagation belongs inside the sampling context, while the parameter update follows it. The optimizer already incorporates its Gaussian prior. Adding the displayed KL, an additional L2 term, or a second regularizer to this loss would change the intended objective. The notebook's supplied sampling helper also restores weight means if a sampling block is interrupted. Its checks verify coherent draws across chunks, positive variances, and preservation of the fitted optimizer state during prediction.

For prediction, sample a weight vector and keep it fixed across all inputs in one function draw:

```
model.eval()                # the IVON model has dropout probability zero
draws = []
with torch.no_grad():
    for _ in range(n_draws):
        with optimizer.sampled_params():
            draws.append(model(x_grid).softmax(dim=-1))
probabilities = torch.stack(draws)  # (T, N, C)
mean_probability = probabilities.mean(dim=0)
```

If prediction is chunked, retain the same context across every chunk of a draw. The resulting tensor has the same meaning as Task 1, so Task 2's entropy and MI calculations apply unchanged. Activating dropout while also sampling IVON weights would define an additional source of randomness that this comparison does not fit or analyze.

13.4 The controlled one-dimensional comparison

The demonstration reuses all 120 observations from the original four-interval example. Both methods use the supplied $1 \to 32 \to 32 \to 2$ tanh network from the small-data acquisition experiment, with

input scaling $x/5$. Within each paired repeat, they start from the same numerical weights. The baseline uses dropout probability 0.2; IVON has dropout probability zero and learns variational weight uncertainty. The original sigmoid-network example in Tasks 1 and 2 remains a separate fit. Compare the two columns of this demonstration with each other, rather than attributing a difference between architectures to posterior inference alone. A training and validation pilot with the original sigmoid architecture left IVON near chance under the 4,000-update budget. The shared tanh architecture fits more effectively within that budget and makes the weight-sampling comparison easier to interpret.

Both receive 4,000 full-batch updates, with one stochastic likelihood-gradient evaluation per update, and 200 model draws for posterior prediction. Equal update and draw counts make these budgets explicit; the two optimizers need not have the same wall-clock cost. Adam uses the supplied one-cycle policy with maximum learning rate 0.03. IVON starts at learning rate 0.1 and uses the multiplier

$$0.05 + 0.95\,\frac{1+\cos(\pi t/4000)}{2}.$$

Its curvature settings are `hess_init=1`, `beta2=0.999`, and `clip_radius=0.1`. Both use regularization coefficient 10^{-4}, and IVON uses `ess=120`. These optimizer policies were fixed using training and validation checks. Their update scales differ, so a shared numerical learning rate would not establish a matched optimization policy. Matching the regularization coefficient alone does not establish equivalent prior distributions for dropout and IVON either.

The three paired repeats use seeds 11, 12, and 13. Only after fitting all six models does the example generate the final 2,048-point evaluation sample with population seed 91027. There is no final-test early stopping or checkpoint selection.

The evaluation population remains the specified mixture of intervals. Predictive accuracy, NLL, and Brier score therefore evaluate inputs on that population's support. Inspect the empty gap and extrapolation through the plots, without assigning them unobserved ground-truth labels. Repeat-level variability describes the stated seeds and fitting policy, rather than uncertainty about every possible BNN method.

Method	Accuracy	NLL	Brier	Fit seconds
Dropout	0.9985 ± 0.0000	0.0130 ± 0.0004	0.0057 ± 0.0001	2.82 ± 0.02
IVON	0.9806 ± 0.0016	0.1578 ± 0.0020	0.0687 ± 0.0020	3.28 ± 0.06

These are means ± sample standard deviations over the three fitting seeds, evaluated on the same 2,048 population observations. Zero displayed standard deviation means zero at the table's rounding precision. The Brier score sums squared probability errors over both classes. Dropout fits this population more accurately under the specified budget. IVON provides a different, explicit weight approximation and visible uncertainty; that construction alone does not ensure improved predictive scores. The fitting times are measurements from this CPU run and exclude posterior prediction. They are not hardware-independent training-time guarantees.

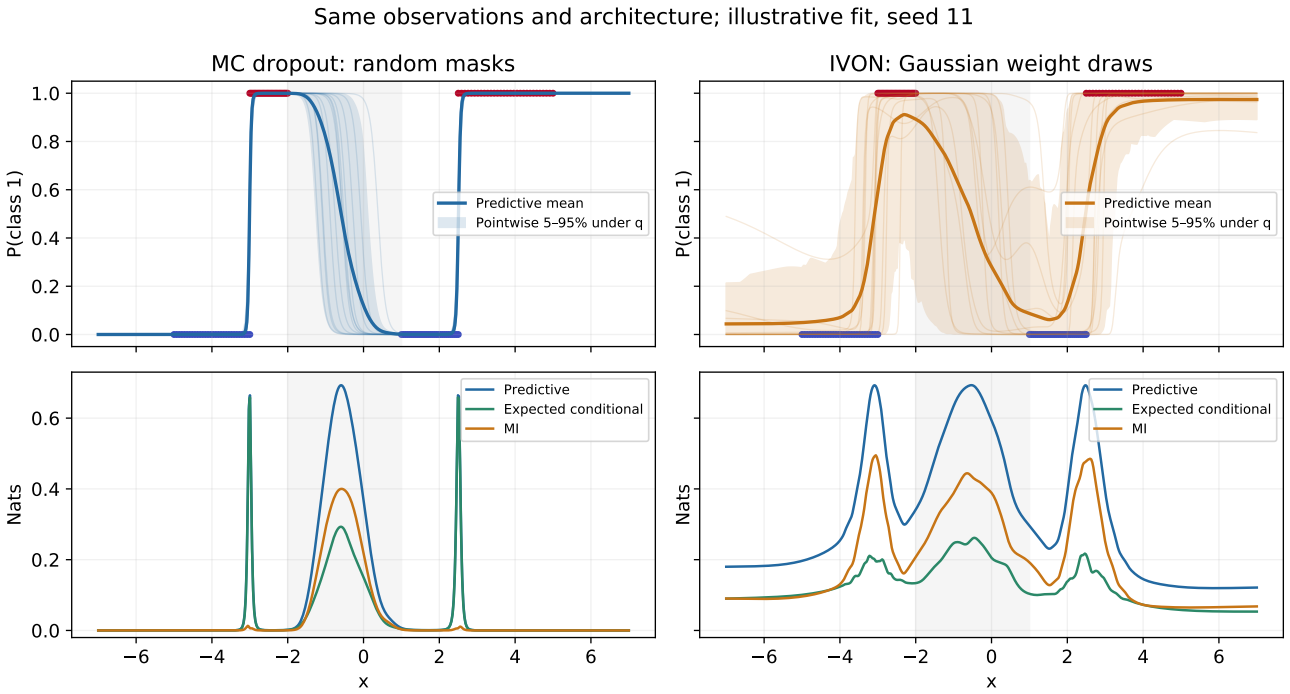


Figure 10: The supplied paired comparison on the original interval example. Each column shows a fitted approximation through sampled probability curves, the predictive mean, and a pointwise 5–95% band, followed by predictive entropy, expected conditional entropy, and MI. The probability band describes variation between sampled models; it is neither a confidence interval for the Monte Carlo mean nor a predictive interval for a binary label.

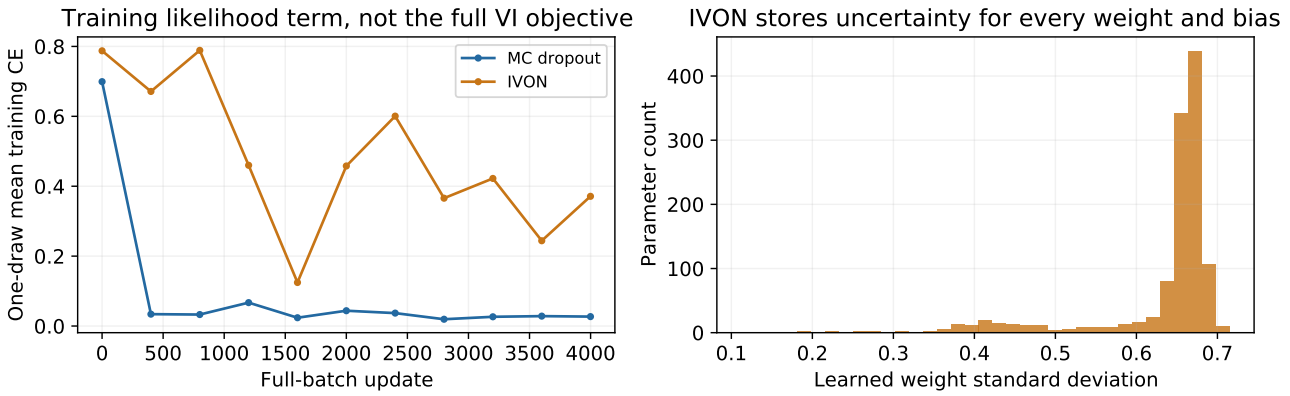


Figure 11: Training diagnostics for the supplied comparison. A sampled training cross entropy is a noisy conditional loss, not the predictive NLL of the probability average. Its plotted likelihood component also omits the variational prior and entropy terms in the full IVON objective. The histogram shows the learned Gaussian standard deviations across weights and biases.

13.5 The supplied GPU image comparison

The image demonstration compares IVON with a dropout CNN using a fixed supervised MNIST training set. It has no acquisition loop and no student implementation task. Its comparison question is how the two fitted approximations predict after seeing the same training labels; Task 4 asks a different question about which labels to acquire.

Uniformly sample 1,000 official training images for fitting and reserve a disjoint 1,000-image validation sample. The two methods use the same CNN as Task 4, with dropout probability 0.5 for Adam and zero for IVON. Each of three paired repeats shares its initial numerical weights and minibatch-index stream. Both methods receive 600 stochastic-gradient evaluations with batch size 64, followed by 30 model draws for prediction. Shared seeds do not make a dropout mask and a Gaussian weight sample the same source of randomness.

Adam uses constant learning rate 0.001 and IVON uses 0.1. Both set `weight_decay` to 10^{-4}. IVON additionally uses `ess=1000`, `hess_init=1`, `beta2=0.99`, and `clip_radius=0.1`; its implied prior precision is 0.1. During development, the IVON learning rates 0.1 and 0.03 and one cosine schedule were checked on validation data. The displayed policy is fixed across repeats, without early stopping or checkpoint selection. This is a small comparison under an announced gradient budget, not an exhaustive search for each method's best possible performance.

Load the official test set only after all six fits are complete. Report accuracy, predictive NLL, multiclass Brier score, and 15-bin ECE on its 10,000 images. The Brier convention here is

$$\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{10}(\widehat{p}_{ic} - \mathbf{1}[y_i = c])^2.$$

For two classes this summed convention equals twice the mean squared error of the class-1 probability. State the convention when quoting a Brier score. The total elapsed time printed by the demonstration includes fitting, validation, and test prediction. It does not measure training time alone.

Set `RUN_IVON=False` to disable the supplied comparisons and `RUN_MNIST=False` to skip their GPU part. The image block requires CUDA and the official `ivon-opt` package. Its dataset cache is shared with Task 4, but the training labels and fitted models are independent of that task's acquisition experiment. Its supplied prediction code also runs when the student exercise functions are unfinished.

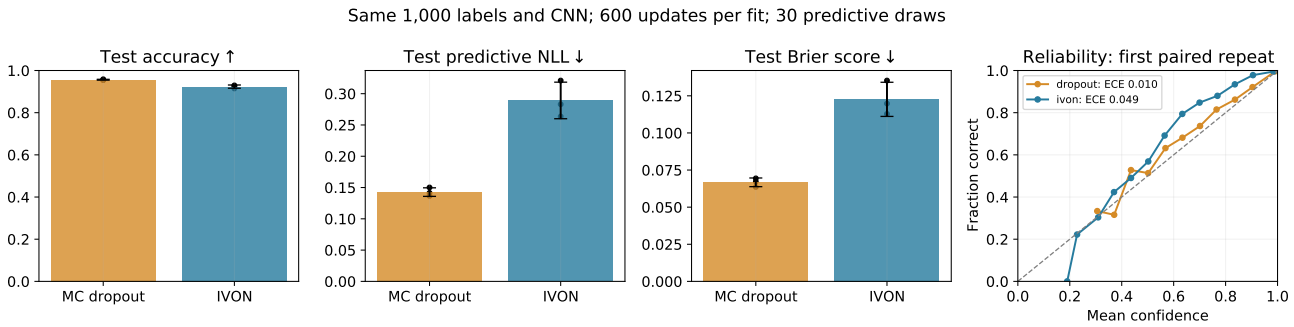


Figure 12: Fixed-label GPU comparison of MC dropout and IVON. Bars and error bars summarize three paired repeats through their means and sample standard deviations, with individual results shown as points. The reliability diagram uses the first paired repeat and the same 15-bin convention as the table.

Method	Accuracy	NLL	Brier	ECE
Dropout	0.9562 ± 0.0020	0.1427 ± 0.0068	0.0667 ± 0.0029	0.0108 ± 0.0012
IVON	0.9236 ± 0.0077	0.2892 ± 0.0292	0.1226 ± 0.0115	0.0491 ± 0.0005

The entries show mean ± sample standard deviation across the three paired repeats. Under this specific training policy, dropout has better average accuracy, NLL, Brier score, and ECE. This provides

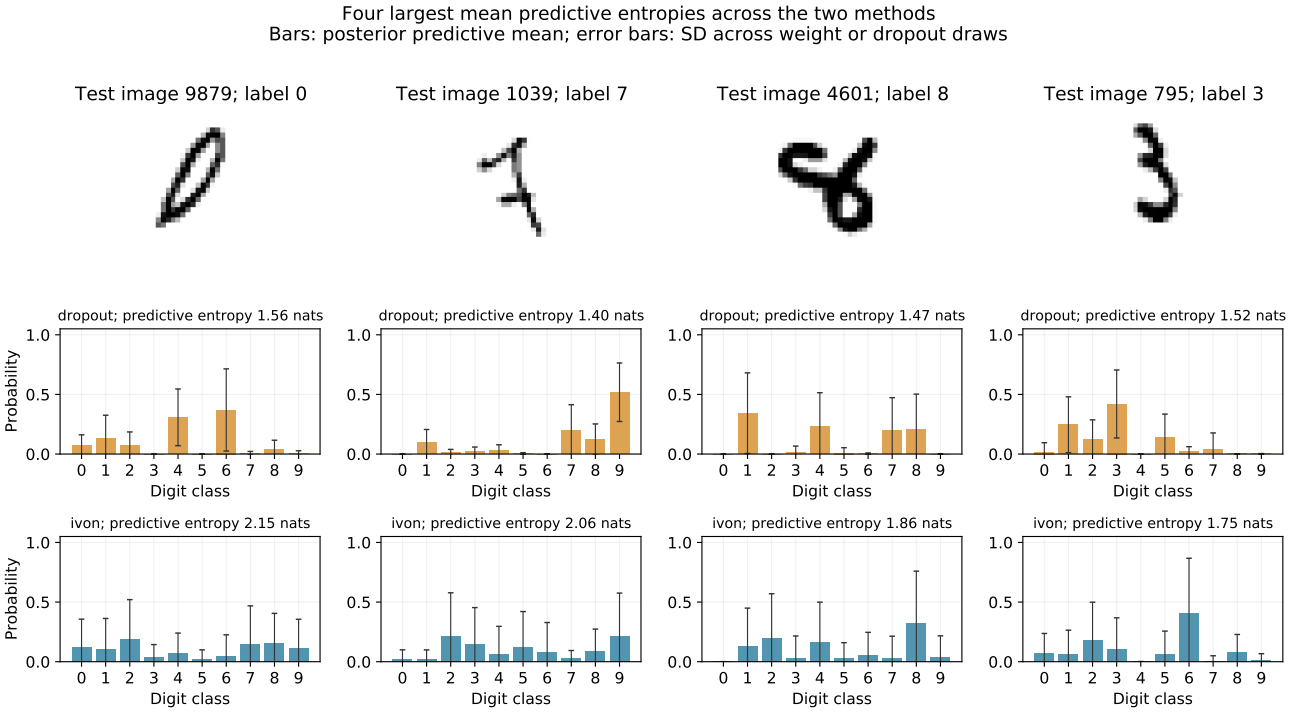


Figure 13: Four MNIST images selected through average predictive entropy across the two methods in the first repeat. Labels are displayed after this probability-based selection. Class-probability bars show how the two fitted approximations distribute their uncertainty on the same image; their error bars show standard deviation across model draws. These are selected illustrations, not an estimate of typical test-set performance.

a useful counterpoint to the method's newer publication date: the fitted model, optimization budget, and chosen prior still determine its practical performance. The conclusion applies to this small, fixed-budget experiment. The complete supplied image comparison took 10.6 seconds on an NVIDIA L40 in the recorded run, including validation and test prediction.

13.6 Discussion prompts and interpretation

Ask students to identify what changes in a model draw: masks in the dropout model, or weight values in IVON. Then ask whether a larger probability band improved predictive NLL or calibration. Both models can have confident extrapolations, and neither a wider band nor a larger MI proves that an approximation is closer to the exact posterior.

Use accuracy, predictive NLL, Brier score, and calibration together. A good class decision can coexist with a poor probability estimate. Prior strength, optimization, the posterior family, and finite Monte Carlo integration can all affect the observed difference. Increasing the draw count addresses only the last of these. The purpose of the supplied example is to expose those distinctions through two concrete fitted models, rather than prescribe one method as a universal replacement for the other.

A Data geometry, intrinsic ambiguity, and unsupported inputs

This optional appendix reuses the supplied toy fit and uncertainty calculations; it adds no implementation task. Enable the appendix experiments only after completing the core tasks.

For overlapping intervals, use class 0 on $[-2, 2]$ and class 1 on $[0, 4]$. For a separate population that chooses either interval with equal probability and samples uniformly inside it, the conditional class-1 probability is $1/2$ in $(0, 2)$. The two intervals have equal lengths, so their population densities agree

there. Outside their overlap but inside the union, only one class has positive density.

The overlap has probability mass 1/2 under this population. Therefore its Bayes classification error is 1/4, and the population-average conditional label entropy is $(\log 2)/2$. These are references for the specified population, not identities that a finite trained dropout network must reproduce exactly. Expected conditional entropy can approach the label ambiguity when the model and approximation fit well; underfitting can also increase that entropy.

For separated intervals, use class 0 on $[-5, -3]$ and class 1 on $[3, 5]$. The reference population has no support in $(-3, 3)$. It does not specify a unique ground-truth label probability in the gap. Inspect the model's extrapolation and disagreement without interpreting either as a known conditional probability of that population. A high-disagreement gap is a possible result of the approximation, not a theorem about dropout.

Changing the dropout rate requires refitting. Increasing it can change regularization, optimization, fit, and the approximation family at once. Do not interpret a wider probability band by itself as evidence that the posterior approximation improved.

B Monte Carlo budgets and redundant acquisition batches

This optional material separates integration precision from statistical information. Compare repeated MC summaries of one fixed fitted model at several draw counts. Expect less integration variability as the count grows, while allowing individual entropy and MI estimates to fluctuate. Their finite-sample nonlinear bias is another reason not to equate a small estimate with the absence of model disagreement.

The draw count and acquisition batch size are different budgets. More draws can improve a pointwise ranking; requesting more top-ranked points can buy redundant labels. For example, several near-identical inputs may all disagree with the model in the same way. Summing marginal information counts that shared distinction repeatedly, whereas joint information considers what additional labels contribute after the first one. The BatchBALD method explicitly addresses joint label information [5]. Its calculation would require coherent model draws across candidate inputs; the ordinary dropout rows in the image practical are only pointwise marginal samples and should not be reused as if they represented a joint function draw.

The core and image practical both use pointwise ranking. They are deliberately transparent about that limitation. A planned extension could study diverse batches or a joint-information approximation, with its own compute budget and repeated comparison against random acquisition. The current test results must not be reused to select such an extension and then reported as its final independent evaluation.

Teaching-material provenance. The original BNN active-learning notebook supplied in this directory credits Ivan Nazarov. The one-dimensional construction is retained from cells 0–9 of the supplied 2025 uncertainty seminar. The new notebooks implement the displayed methods locally and keep both reference notebooks unchanged.

References

[1] Yuesong Shen and colleagues. *Variational learning is effective for large deep networks.* ICML, 2024. PMLR 235:44665–44686. Official optimizer and usage examples.

[2] Yarin Gal and Zoubin Ghahramani. *Dropout as a Bayesian approximation: representing model uncertainty in deep learning.* ICML, 2016. PMLR 48:1050–1059.

[3] Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. *Bayesian active learning for classification and preference learning.* 2011. arXiv:1112.5745.

[4] Yarin Gal, Riashat Islam, and Zoubin Ghahramani. *Deep Bayesian active learning with image data.* ICML, 2017. PMLR 70:1183–1192.

[5] Andreas Kirsch, Joost van Amersfoort, and Yarin Gal. *BatchBALD: efficient and diverse batch acquisition for deep Bayesian active learning*. NeurIPS, 2019. Paper.